

ESTUDO DIRIGIDO – VETORES E LISTAS

ALUNO: Mária Lais MarquesPassos

Parte I – Questões conceituais

1. O que é uma estrutura de dados?

Estruturas de dados são maneiras de organizar informações para que um programa possa armazená-las, acessá-las e manipulá-las de forma eficiente. Um exemplo é o vetor, usado para armazenar vários elementos do mesmo tipo.

2. Explique o que é um vetor e como seus elementos são organizados.

Um vetor (array) é uma estrutura que armazena vários elementos do mesmo tipo, organizados sequencialmente e identificados por meio de um índice.

3. Considere: `int valores[5] = {4, 8, 12, 16, 20};`

- a) `valores[0] = 4`.
- b) `valores[3] = 16`.
- c) O número 20 está no índice 4.
- d) O vetor possui 5 elementos.

4. Diferença entre vetor e lista encadeada.

O vetor organiza os elementos sequencialmente e os identifica por índices. A lista encadeada pode aumentar ou diminuir durante a execução do programa, conforme apresentado no material de aula.

5. O que é um nó em uma lista encadeada?

Um nó é uma unidade usada para formar uma lista encadeada. Ele armazena um elemento e uma referência para o próximo elemento da sequência.

6. Função do ponteiro próximo.

O ponteiro próximo indica qual é o próximo nó da lista, permitindo que os elementos sejam percorridos em sequência.

7. Por que o último nó aponta para NULL ou `nullptr`?

Para indicar que não existe outro nó depois dele, marcando o final da lista.

8. Duas vantagens do vetor e duas da lista encadeada.

Vetor: acesso aos elementos por índice e organização sequencial. Lista encadeada: pode aumentar ou diminuir durante a execução e permite organizar uma sequência de elementos com inserções e remoções.

Parte II – Exercícios com Vetores

9. Preenchimento e exibição

```
#include <iostream>
using namespace std;

int main() {
    int v[10];

    for(int i = 0; i < 10; i++)
        cin >> v[i];

    for(int i = 0; i < 10; i++)
        cout << v[i] << " ";

    return 0;
```

```
}
```

10. Maior e menor valor

```
#include <iostream>
using namespace std;

int main() {
    int v[10];
    for (int i = 0; i < 10; i++) cin >> v[i];

    int maior=v[0], menor=v[0];
    int posMaior=0, posMenor=0;

    for(int i=1; i<10; i++) {
        if (v[i] > maior) { maior=v[i]; posMaior=i; }
        if (v[i] < menor) { menor=v[i]; posMenor=i; }
    }

    cout << "Maior: " << maior << " Posicao: " << posMaior << endl;
    cout << "Menor: " << menor << " Posicao: " << posMenor << endl;
    return 0;
}
```

11. Média das notas

```
#include <iostream>
using namespace std;

int main() {
    double n[5], soma=0;

    for(int i=0; i<5; i++) {
        cin >> n[i];
        soma += n[i];
    }

    double media=soma/5, maior=n[0], menor=n[0];
    int acima=0;

    for(int i=1; i<5; i++) {
        if (n[i]>maior) maior=n[i];
        if (n[i]<menor) menor=n[i];
    }

    for(int i=0; i<5; i++)
        if (n[i]>media) acima++;

    cout << "Media: " << media << endl;
    cout << "Maior: " << maior << endl;
    cout << "Menor: " << menor << endl;
    cout << "Acima da media: " << acima << endl;
    return 0;
}
```

12. Pares e ímpares

```
#include <iostream>
using namespace std;

int main() {
    int v[10], pares=0, impares=0;

    for(int i=0; i<10; i++) {
        cin >> v[i];
        if (v[i]%2==0) pares++;
        else impares++;
    }

    cout << "Pares: " << pares << endl;
    cout << "Impares: " << impares << endl;
    return 0;
}
```

13. Busca em vetor

```
#include <iostream>
using namespace std;

int main() {
```

```

int v[10], valor;
bool encontrado=false;

for (int i=0; i<10; i++) cin >> v[i];
cin >> valor;

for(inti=0; i<10; i++) {
    if(v[i]==valor) {
        cout << "Valor encontrado na posicao " << i << endl;
        encontrado=true;
        break;
    }
}

if(!encontrado)
    cout << "Valor nao encontrado" << endl;

return 0;
}

```

Parte III – Exercícios com Listas Encadeadas

14. Identificando os elementos

- a) Primeiro elemento: 10.
- b) Último elemento: 45.
- c) Depois de 25 vem 30.
- d) Existem 4 nós.
- e) O nó 45 aponta para NULL.

15. Inserção em uma lista

Lista inicial: 10 → 20 → 30 → NULL

Inserindo 5 no início:

5 → 10 → 20 → 30 → NULL

Inserindo 40 no final:

5 → 10 → 20 → 30 → 40 → NULL

16. Remoção em uma lista

Inicial: 5 → 10 → 20 → 30 → 40 → NULL

- a) Após remover 5: 10 → 20 → 30 → 40 → NULL
- b) Após remover 30: 10 → 20 → 40 → NULL
- c) Após remover 40: 10 → 20 → NULL

17. Inserção no início em C++

```

void inserirInicio(No*& inicio, int valor) {
    No* novo = new No;
    novo->valor = valor;
    novo->proximo = inicio;
    inicio = novo;
}

```

18. Percorrendo uma lista

```

void percorrer(No* inicio) {
    No* atual = inicio;

    while (atual != nullptr) {
        cout << atual->valor << " ";
        atual = atual->proximo;
    }
}

```

19. Busca em lista

```

bool buscar(No* inicio, int valor) {
    No* atual = inicio;

    while (atual != nullptr) {
        if(atual->valor == valor)
            return true;

        atual = atual->proximo;
    }

    return false;
}

```

Parte IV – Desafios

20. Sistema de notas com vetor

```

#include <iostream>
using namespace std;

int main() {
    double notas[10], soma=0, procurada;

    for(int i=0; i<10; i++) {
        cin >> notas[i];
        soma += notas[i];
    }

    double media=soma/10;
    double maior=notas[0], menor=notas[0];
    int acima=0;

    for(int i=1; i<10; i++) {
        if (notas[i]>maior) maior=notas[i];
        if (notas[i]<menor) menor=notas[i];
    }

    for(int i=0; i<10; i++)
        if (notas[i]>media) acima++;

    cout << "Notas: ";
    for (int i=0; i<10; i++) cout << notas[i] << " ";

    cout << "\nMedia: " << media;
    cout << "\nMaior: " << maior;
    cout << "\nMenor: " << menor;
    cout << "\nAcima da media: " << acima << endl;

    cout << "Digite uma nota para procurar: ";
    cin >> procurada;

    bool achou=false;
    for(int i=0; i<10; i++) {
        if(notas[i]==procurada) {
            cout << "Nota encontrada na posicao " << i << endl;
            achou=true;
        }
    }

    if (!achou) cout << "Nota nao encontrada" << endl;

    return 0;
}

```

21. Cadastro utilizando lista encadeada

```

#include <iostream>
using namespace std;

struct No {
    int valor;
    No* proximo;
};

void inserir(No*& inicio, int valor) {
    No* novo = new No(valor, inicio);
    inicio = novo;
}

```

```

void listar(No* inicio) {
    while (inicio != nullptr) {
        cout << inicio->valor << " ";
        inicio = inicio->proximo;
    }
    cout << endl;
}

bool buscar(No* inicio, int valor) {
    while (inicio != nullptr) {
        if (inicio->valor == valor) return true;
        inicio = inicio->proximo;
    }
    return false;
}

void remover(No*& inicio, int valor) {
    No* atual=inicio;
    No* anterior=nullptr;

    while (atual != nullptr && atual->valor != valor) {
        anterior=atual;
        atual=atual->proximo;
    }

    if (atual == nullptr) return;

    if(anterior == nullptr)
        inicio=atual->proximo;
    else
        anterior->proximo=atual->proximo;

    delete atual;
}

int contar(No* inicio) {
    int total=0;
    while (inicio != nullptr) {
        total++;
        inicio=inicio->proximo;
    }
    return total;
}

int main() {
    No* inicio=nullptr;
    int opcao, valor;

    do{
        cout << "\n1 - Inserir elemento";
        cout << "\n2 - Listar elementos";
        cout << "\n3 - Buscar elemento";
        cout << "\n4 - Remover elemento";
        cout << "\n5 - Contar elementos";
        cout << "\n0 - Sair\n";
        cin >> opcao;

        switch(opcao) {
            case 1:
                cin >> valor;
                inserir(inicio, valor);
                break;
            case 2:
                listar(inicio);
                break;
            case 3:
                cin >> valor;
                cout << (buscar(inicio, valor) ? "Encontrado" : "Nao encontrado") << endl;
                break;
            case 4:
                cin >> valor;
                remover(inicio, valor);
                break;
            case 5:
                cout << "Quantidade: " << contar(inicio) << endl;
                break;
        }
    } while(opcao != 0);

    return 0;
}

```

}

Parte V – Questão de análise

Considerando o material, o vetor organiza os elementos sequencialmente e usa índices para identificá-los. Já a lista encadeada pode aumentar ou diminuir durante a execução. Portanto, quando a quantidade de registros muda constantemente e há inserções e remoções, a lista encadeada é uma opção adequada. O vetor tem como vantagem o acesso por índice; sua limitação, nesse cenário, é ter uma organização de tamanho menos flexível. A lista encadeada é flexível quanto ao tamanho; em contrapartida, o acesso aos elementos é feito percorrendo a sequência.

Quadro para síntese

Característica	Vetor	Lista Encadeada
Tamanho	organização de tamanho definido	pode aumentar ou diminuir
Acesso aos elementos	Acesso por índice	acesso percorrendo a sequência de nós
Inserção/remoção	pode exigir reorganização/deslocamento de elementos	Mais flexíveis para inserções e remoções
Uso de memória	elementos organizados sequencialmente	Cada nó armazena o elemento e uma referência para o próximo
Exemplo de aplicação	Armazenar notas de alunos	armazenar serviços que pode aumentar ou diminuir